

Guide Complet de Java Orienté Objet – ESPRIT

Étudiant:	Mohamed Chahbani
Classe:	3A10
École:	ESPRIT
Année:	2024-2025

Un guide exhaustif pour maîtriser la programmation orientée objet en Java

Table des matières

[Fondamentaux Java](#1-fondamentaux-java)

[POO : Les 4 piliers](#2-poo-les-4-piliers)

[Classes, objets, constructeurs](#3-classes-objets-constructeurs)

[Encapsulation avancée](#4-encapsulation-avancée)

[Héritage](#5-héritage)

[Polymorphisme](#6-polymorphisme)

[Interfaces](#7-interfaces)

[Classes abstraites](#8-classes-abstraites)

[Exceptions](#9-exceptions)

[Collections (List, Set, Map)](#10-collections-list-set-map)

[Interfaces fonctionnelles & Lambda](#11-interfaces-fonctionnelles--lambda)

[API Stream](#12-api-stream)

[Patterns courants](#13-patterns-courants)

1. Fondamentaux Java

■ 1.1 La Plateforme Java

Java s'exécute sur une **machine virtuelle** (JVM) pour assurer la portabilité multi-plateforme :

Composant	Description
JDK	Kit de développement complet (compilateur `javac`, outils de debug)
JRE	Runtime Environment (JVM + bibliothèques standard)
JVM	Machine virtuelle qui exécute le bytecode `.class`

Cycle de compilation et exécution :

JAVA	Exemple de code
1	fichier.java → javac → fichier.class → JVM → Exécution

■ 1.2 Structure d'un programme Java

JAVA	Exemple de code
1	public class HelloWorld {
2	public static void main(String[] args) {
3	System.out.println("Hello 3A");
4	}
5	}

■ Règles essentielles :

- Une seule classe `public` par fichier
- Le nom du fichier doit correspondre au nom de la classe publique
- La méthode `main(String[] args)` est le point d'entrée du programme
- Les arguments de ligne de commande sont accessibles via le tableau `args`

■ 1.3 Types de données et variables

■ Types primitifs

JAVA	Exemple de code
1	// Entiers
2	byte b = 127; // 8 bits (-128 à 127)
3	short s = 32000; // 16 bits (-32,768 à 32,767)
4	int x = 10; // 32 bits (type par défaut)
5	long y = 100000000000L; // 64 bits (suffixe L obligatoire)
6	// Réels
7	float f = 3.14f; // 32 bits (suffixe f obligatoire)
8	double z = 3.14159265359; // 64 bits (type par défaut)
9	// Autres
10	boolean b = true; // true ou false
11	char c = 'A'; // caractère Unicode (16 bits)

■ Types référence (objets)

JAVA	Exemple de code
1	String s = "Hello World";
2	int[] tableau = {1, 2, 3, 4, 5};
3	ArrayList<Integer> liste = new ArrayList<>();

■ 1.4 Opérateurs et structures de contrôle

■ Opérateurs arithmétiques

JAVA	Exemple de code
1	<code>int resultat = 10 + 5 * 2; // 20 (priorité: * avant +)</code>
2	<code>int division = 10 / 3; // 3 (division entière)</code>
3	<code>int modulo = 10 % 3; // 1 (reste de la division)</code>

■ Structures conditionnelles

JAVA	Exemple de code
1	<code>// if-else</code>
2	<code>if (age >= 18) {</code>
3	<code>System.out.println("Majeur");</code>
4	<code>} else {</code>
5	<code>System.out.println("Mineur");</code>
6	<code>}</code>
7	<code>// switch (Java 14+)</code>
8	<code>String jour = switch (dayOfWeek) {</code>
9	<code>case 1 -> "Lundi";</code>
10	<code>case 2 -> "Mardi";</code>
11	<code>default -> "Autre jour";</code>
12	<code>};</code>

■ Boucles

JAVA	Exemple de code
1	// for classique
2	for (int i = 0; i < 10; i++) {
3	System.out.println(i);
4	}
5	// for-each
6	int[] nombres = {1, 2, 3, 4, 5};
7	for (int nombre : nombres) {
8	System.out.println(nombre);
9	}
10	// while
11	while (condition) {
12	// code
13	}
14	// do-while
15	do {
16	// code exécuté au moins une fois
17	} while (condition);

2. POO : Les 4 piliers

■ 2.1 Encapsulation

- Principe : Masquer les détails d'implémentation et exposer uniquement une interface publique contrôlée.

JAVA	Exemple de code
1	public class CompteBancaire {
2	private double solde; // Attribut privé
3	private String titulaire;
4	public CompteBancaire(String titulaire, double soldeInitial) {
5	this.titulaire = titulaire;
6	this.solde = soldeInitial;
7	}
8	// Getter avec logique métier
9	public double getSolde() {
10	return solde;
11	}
12	// Setter avec validation
13	public void deposer(double montant) {
14	if (montant > 0) {
15	this.solde += montant;
16	} else {
17	throw new IllegalArgumentException("Montant invalide");
18	}
19	}
20	public void retirer(double montant) {
21	if (montant > 0 && montant <= solde) {
22	this.solde -= montant;
23	} else {
24	throw new IllegalArgumentException("Retrait impossible");
25	}
26	}
27	}

✓ Avantages :

- ✓ Contrôle d'accès aux données
- ✓ Validation des données avant modification
- ✓ Flexibilité : possibilité de changer l'implémentation interne sans affecter le code client

■ 2.2 Héritage

■ Principe : Réutilisation et spécialisation du code existant.

JAVA	Exemple de code
1	public class Animal {
2	protected String nom;
3	protected int age;
4	public Animal(String nom, int age) {
5	this.nom = nom;
6	this.age = age;
7	}
8	public void manger() {
9	System.out.println(nom + " mange");
10	}
11	public void dormir() {
12	System.out.println(nom + " dort");
13	}
14	}
15	public class Chien extends Animal {
16	private String race;
17	public Chien(String nom, int age, String race) {
18	super(nom, age); // Appel du constructeur parent
19	this.race = race;
20	}
21	public void aboyer() {
22	System.out.println(nom + " aboie: Ouaf!");
23	}
24	@Override
25	public void manger() {
26	System.out.println(nom + " mange des croquettes");
27	}
28	}
29	// Utilisation

```
30 Chien monChien = new Chien("Rex", 3, "Labrador");
31 monChien.manger(); // "Rex mange des croquettes" (méthode redéfinie)
32 monChien.dormir(); // "Rex dort" (méthode héritée)
33 monChien.aboyer(); // "Rex aboie: Ouaf!" (méthode propre)
```

Points clés :

- Une classe ne peut hériter que d'**une seule** classe (pas d'héritage multiple)
- Toutes les classes héritent implicitement de `Object`
- Mot-clé `super` pour accéder aux membres de la classe parente
- Mot-clé `final` pour empêcher l'héritage : `public final class MaClasse`

■ 2.3 Polymorphisme

■ Principe : Un même appel de méthode peut avoir des comportements différents selon le type réel de l'objet.

JAVA	Exemple de code
1	public class Forme {
2	public void dessiner() {
3	System.out.println("Dessine une forme");
4	}
5	public double calculerAire() {
6	return 0.0;
7	}
8	}
9	public class Cercle extends Forme {
10	private double rayon;
11	public Cercle(double rayon) {
12	this.rayon = rayon;
13	}
14	@Override
15	public void dessiner() {
16	System.out.println("Dessine un cercle");
17	}
18	@Override
19	public double calculerAire() {
20	return Math.PI * rayon * rayon;
21	}
22	}
23	public class Rectangle extends Forme {
24	private double longueur, largeur;
25	public Rectangle(double longueur, double largeur) {
26	this.longueur = longueur;
27	this.largeur = largeur;
28	}
29	@Override

```
30     public void dessiner() {
31         System.out.println("Dessine un rectangle");
32     }
33     @Override
34     public double calculerAire() {
35         return longueur * largeur;
36     }
37 }
38 // Démonstration du polymorphisme
39 public class Demo {
40     public static void main(String[] args) {
41         Forme[] formes = {
42             new Cercle(5),
43             new Rectangle(4, 6),
44             new Cercle(3)
45         };
46         for (Forme forme : formes) {
47             forme.dessiner(); // Appel polymorphe
48             System.out.println("Aire: " + forme.calculerAire());
49         }
50     }
51 }
```

Types de polymorphisme :

****Polymorphisme d'inclusion**** : via l'héritage (exemple ci-dessus)

****Polymorphisme paramétrique**** : via les génériques (`List`)

****Surcharge**** : plusieurs méthodes avec le même nom mais signatures différentes

■ 2.4 Abstraction

■ Principe : Définir un contrat sans fournir tous les détails d'implémentation.

JAVA	Exemple de code
1	public abstract class Vehicule {
2	protected String marque;
3	protected int annee;
4	public Vehicule(String marque, int annee) {
5	this.marque = marque;
6	this.annee = annee;
7	}
8	// Méthode abstraite (pas d'implémentation)
9	public abstract void demarrer();
10	public abstract void arreter();
11	// Méthode concrète
12	public void afficherInfo() {
13	System.out.println("Véhicule: " + marque + " (" + annee + ")");
14	}
15	}
16	public class Voiture extends Vehicule {
17	public Voiture(String marque, int annee) {
18	super(marque, annee);
19	}
20	@Override
21	public void demarrer() {
22	System.out.println("Le moteur démarre");
23	}
24	@Override
25	public void arreter() {
26	System.out.println("Le moteur s'arrête");
27	}
28	}
29	// ERREUR : impossible d'instancier une classe abstraite

```
30 // Vehicule v = new Vehicule("Toyota", 2024);  
31 // CORRECT  
32 Vehicule v = new Voiture("Toyota", 2024);  
33 v.demarrer();
```

3. Classes, objets, constructeurs

■ 3.1 Déclaration d'une classe complète

JAVA	Exemple de code
1	public class Patient {
2	// Attributs (état de l'objet)
3	private int cin;
4	private String nom;
5	private String prenom;
6	private int numSecuriteSociale;
7	private LocalDate dateNaissance;
8	// Constructeur complet
9	public Patient(int cin, String nom, String prenom,
10	int numSecuriteSociale, LocalDate dateNaissance) {
11	this.cin = cin;
12	this.nom = nom;
13	this.prenom = prenom;
14	this.numSecuriteSociale = numSecuriteSociale;
15	this.dateNaissance = dateNaissance;
16	}
17	// Getters
18	public int getCin() { return cin; }
19	public String getNom() { return nom; }
20	public String getPrenom() { return prenom; }
21	public int getNumSecuriteSociale() { return numSecuriteSociale; }
22	public LocalDate getDateNaissance() { return dateNaissance; }
23	// Setters
24	public void setNom(String nom) {
25	if (nom != null && !nom.isEmpty()) {
26	this.nom = nom;
27	}
28	}
29	public void setPrenom(String prenom) {

```
30     if (prenom != null && !prenom.isEmpty()) {
31         this.prenom = prenom;
32     }
33 }
34 // Méthode métier
35 public int calculerAge() {
36     return Period.between(dateNaissance, LocalDate.now()).getYears();
37 }
38 }
```

■ 3.2 Constructeurs et surcharge

JAVA	Exemple de code
1	public class Voiture {
2	private String marque;
3	private String modele;
4	private int annee;
5	private double prix;
6	// Constructeur complet
7	public Voiture(String marque, String modele, int annee, double prix) {
8	this.marque = marque;
9	this.modele = modele;
10	this.annee = annee;
11	this.prix = prix;
12	}
13	// Constructeur avec valeur par défaut pour le prix
14	public Voiture(String marque, String modele, int annee) {
15	this(marque, modele, annee, 0.0); // Délégation
16	}
17	// Constructeur minimal
18	public Voiture(String marque, String modele) {
19	this(marque, modele, LocalDate.now().getYear());
20	}
21	// Constructeur par défaut
22	public Voiture() {
23	this("Inconnue", "Inconnu", LocalDate.now().getYear(), 0.0);
24	}
25	}
26	// Utilisation
27	Voiture v1 = new Voiture("Toyota", "Corolla", 2024, 25000);
28	Voiture v2 = new Voiture("Honda", "Civic", 2024);
29	Voiture v3 = new Voiture("Ford", "Focus");

30

```
Voiture v4 = new Voiture();
```

■ Règles importantes :

- Le constructeur a le même nom que la classe
- Pas de type de retour (même pas `void`)
- Premier appel doit être `this(...)` ou `super(...)`
- Si aucun constructeur n'est défini, Java crée un constructeur par défaut

■ 3.3 Mot-clé `this`

JAVA	Exemple de code
1	public class Personne {
2	private String nom;
3	private int age;
4	// 1. Distinguer attribut et paramètre
5	public Personne(String nom, int age) {
6	this.nom = nom; // this.nom = attribut, nom = paramètre
7	this.age = age;
8	}
9	// 2. Retourner l'objet courant (pattern fluent)
10	public Personne setNom(String nom) {
11	this.nom = nom;
12	return this; // Permet le chaînage
13	}
14	public Personne setAge(int age) {
15	this.age = age;
16	return this;
17	}
18	}
19	// Utilisation avec chaînage
20	Personne p = new Personne("Ali", 25)
21	.setNom("Ahmed")
22	.setAge(30);

■ 3.4 Mot-clé `static`

JAVA	Exemple de code
1	public class Compteur {
2	// Variable de classe (partagée par toutes les instances)
3	private static int nombreInstances = 0;
4	// Variable d'instance (propre à chaque objet)
5	private int id;
6	public Compteur() {
7	nombreInstances++;
8	this.id = nombreInstances;
9	}
10	// Méthode de classe
11	public static int getNombreInstances() {
12	return nombreInstances;
13	}
14	// Méthode d'instance
15	public int getId() {
16	return id;
17	}
18	// Bloc static (exécuté une seule fois au chargement de la classe)
19	static {
20	System.out.println("Classe Compteur chargée");
21	}
22	}
23	// Utilisation
24	System.out.println(Compteur.getNombreInstances()); // 0
25	Compteur c1 = new Compteur();
26	Compteur c2 = new Compteur();
27	Compteur c3 = new Compteur();
28	System.out.println(Compteur.getNombreInstances()); // 3
29	System.out.println(c1.getId()); // 1

30

```
System.out.println(c2.getId()); // 2
```

4. Encapsulation avancée

■ 4.1 Redéfinition de `toString`, `equals`, `hashCode`

JAVA	Exemple de code
1	<code>import java.util.Objects;</code>
2	<code>public class Patient {</code>
3	<code> private int cin;</code>
4	<code> private String nom;</code>
5	<code> private String prenom;</code>
6	<code> private int numSecuriteSociale;</code>
7	<code> public Patient(int cin, String nom, String prenom, int numSecuriteSociale) {</code>
8	<code> this.cin = cin;</code>
9	<code> this.nom = nom;</code>
10	<code> this.prenom = prenom;</code>
11	<code> this.numSecuriteSociale = numSecuriteSociale;</code>
12	<code> }</code>
13	<code> // toString : représentation textuelle lisible</code>
14	<code> @Override</code>
15	<code> public String toString() {</code>
16	<code> return String.format("Patient{cin=%d, nom='%s', prenom='%s', numSS=%d}",</code>
17	<code> cin, nom, prenom, numSecuriteSociale);</code>
18	<code> }</code>
19	<code> // equals : définit l'égalité logique</code>
20	<code> @Override</code>
21	<code> public boolean equals(Object obj) {</code>
22	<code> if (this == obj) return true; // Même référence</code>
23	<code> if (obj == null getClass() != obj.getClass()) return false;</code>
24	<code> Patient other = (Patient) obj;</code>
25	<code> return cin == other.cin &&</code>
26	<code> numSecuriteSociale == other.numSecuriteSociale;</code>
27	<code> }</code>
28	<code> // hashCode : DOIT être cohérent avec equals</code>
29	<code> @Override</code>

```
30     public int hashCode() {  
31         return Objects.hash(cin, numSecuriteSociale);  
32     }  
33 }
```

Importance :

- `toString()` : facilite le débogage et l'affichage
- `equals()` : compare le contenu logique (vs `==` qui compare les références)
- `hashCode()` : essentiel pour les collections `HashSet`, `HashMap`
- **Règle d'or** : Si `a.equals(b)` est `true`, alors `a.hashCode() == b.hashCode()`

■ 4.2 Immutabilité

JAVA	Exemple de code
1	<code>public final class PointImmutable {</code>
2	<code> private final int x;</code>
3	<code> private final int y;</code>
4	<code> public PointImmutable(int x, int y) {</code>
5	<code> this.x = x;</code>
6	<code> this.y = y;</code>
7	<code> }</code>
8	<code> public int getX() { return x; }</code>
9	<code> public int getY() { return y; }</code>
10	<code> // Pas de setters !</code>
11	<code> // Méthodes qui retournent de nouvelles instances</code>
12	<code> public PointImmutable deplacer(int dx, int dy) {</code>
13	<code> return new PointImmutable(x + dx, y + dy);</code>
14	<code> }</code>
15	<code>}</code>
16	<code>// Utilisation</code>
17	<code>PointImmutable p1 = new PointImmutable(0, 0);</code>
18	<code>PointImmutable p2 = p1.deplacer(5, 3); // p1 reste inchangé</code>

✓ Avantages de l'immuabilité :

- ✓ Thread-safe par nature
- ✓ Utilisable comme clé dans les `HashMap`
- ✓ Comportement prévisible

5. Héritage

■ 5.1 Héritage simple

JAVA	Exemple de code
1	public class Employe {
2	protected String nom;
3	protected String prenom;
4	protected double salaire;
5	public Employe(String nom, String prenom, double salaire) {
6	this.nom = nom;
7	this.prenom = prenom;
8	this.salaire = salaire;
9	}
10	public double calculerSalaire() {
11	return salaire;
12	}
13	public void afficher() {
14	System.out.println(nom + " " + prenom);
15	}
16	}
17	public class Manager extends Employe {
18	private double prime;
19	private List<Employe> equipe;
20	public Manager(String nom, String prenom, double salaire, double prime) {
21	super(nom, prenom, salaire);
22	this.prime = prime;
23	this.equipe = new ArrayList<>();
24	}
25	@Override
26	public double calculerSalaire() {
27	return super.calculerSalaire() + prime;
28	}
29	@Override


```
30     public void afficher() {  
31         super.afficher();  
32         System.out.println("Manager - Équipe: " + equipe.size() + " personnes");  
33     }  
34     public void ajouterMembre(Employe e) {  
35         equipe.add(e);  
36     }  
37 }
```

■ 5.2 Constructeurs et héritage

JAVA	Exemple de code
1	public class Personne {
2	private String nom;
3	private int age;
4	public Personne(String nom, int age) {
5	this.nom = nom;
6	this.age = age;
7	System.out.println("Constructeur Personne");
8	}
9	}
10	public class Etudiant extends Personne {
11	private String matricule;
12	public Etudiant(String nom, int age, String matricule) {
13	super(nom, age); // DOIT être la première instruction
14	this.matricule = matricule;
15	System.out.println("Constructeur Etudiant");
16	}
17	}
18	// Création
19	Etudiant e = new Etudiant("Ali", 20, "E12345");
20	// Affiche:
21	// Constructeur Personne
22	// Constructeur Etudiant

■ 5.3 Modificateurs d'accès

Modificateur	Classe	Package	Sous-classe	Monde
--------------	--------	---------	-------------	-------

`private`	■	■	■	■
(défaut)	■	■	■	■
`protected`	■	■	■	■
`public`	■	■	■	■

JAVA	Exemple de code
1	<code>public class Parent {</code>
2	<code>private int prive; // Accessible uniquement dans Parent</code>
3	<code>int default; // Accessible dans le package</code>
4	<code>protected int protege; // Accessible dans les sous-classes</code>
5	<code>public int public; // Accessible partout</code>
6	<code>}</code>
7	<code>public class Enfant extends Parent {</code>
8	<code>public void methode() {</code>
9	<code>// prive = 10; // ERREUR</code>
10	<code>default = 10; // OK si même package</code>
11	<code>protege = 10; // OK</code>
12	<code>public = 10; // OK</code>
13	<code>}</code>
14	<code>}</code>

6. Polymorphisme

■ 6.1 Polymorphisme d'inclusion (runtime)

JAVA	Exemple de code
1	public class Animal {
2	public void faireDuBruit() {
3	System.out.println("L'animal fait du bruit");
4	}
5	}
6	public class Chien extends Animal {
7	@Override
8	public void faireDuBruit() {
9	System.out.println("Ouaf ouaf!");
10	}
11	}
12	public class Chat extends Animal {
13	@Override
14	public void faireDuBruit() {
15	System.out.println("Miaou!");
16	}
17	}
18	// Polymorphisme en action
19	Animal[] animaux = {
20	new Chien(),
21	new Chat(),
22	new Animal()
23	};
24	for (Animal animal : animaux) {
25	animal.faireDuBruit(); // Méthode appelée selon le type réel
26	}
27	// Affiche:
28	// Ouaf ouaf!
29	// Miaou!

```
30 // L'animal fait du bruit
```

■ 6.2 Surcharge (polymorphisme compile-time)

JAVA	Exemple de code
1	<code>public class Calculatrice {</code>
2	<code>// Même nom, signatures différentes</code>
3	<code>public int additionner(int a, int b) {</code>
4	<code>return a + b;</code>
5	<code>}</code>
6	<code>public double additionner(double a, double b) {</code>
7	<code>return a + b;</code>
8	<code>}</code>
9	<code>public int additionner(int a, int b, int c) {</code>
10	<code>return a + b + c;</code>
11	<code>}</code>
12	<code>public String additionner(String a, String b) {</code>
13	<code>return a + b;</code>
14	<code>}</code>
15	<code>}</code>
16	<code>// Utilisation</code>
17	<code>Calculatrice calc = new Calculatrice();</code>
18	<code>System.out.println(calc.additionner(5, 3)); // 8</code>
19	<code>System.out.println(calc.additionner(5.5, 3.2)); // 8.7</code>
20	<code>System.out.println(calc.additionner(5, 3, 2)); // 10</code>
21	<code>System.out.println(calc.additionner("Hello", "World")); // HelloWorld</code>

■ 6.3 Casting et instanceof

JAVA	Exemple de code
1	<code>Animal animal = new Chien(); // Upcasting (implicite)</code>
2	<code>// instanceof : vérifier le type</code>
3	<code>if (animal instanceof Chien) {</code>
4	<code>Chien chien = (Chien) animal; // Downcasting (explicite)</code>
5	<code>chien.aboyer();</code>
6	<code>}</code>
7	<code>// Java 16+ : pattern matching</code>
8	<code>if (animal instanceof Chien chien) {</code>
9	<code>chien.aboyer(); // Variable 'chien' automatiquement créée</code>
10	<code>}</code>

7. Interfaces

■ 7.1 Déclaration et implémentation

JAVA	Exemple de code
1	<code>public interface Dessinable {</code>
2	<code>// Méthodes abstraites (implicitement public abstract)</code>
3	<code>void dessiner();</code>
4	<code>void effacer();</code>
5	<code>// Constantes (implicitement public static final)</code>
6	<code>int EPAISSEUR_DEFAULT = 1;</code>
7	<code>// Méthode par défaut (Java 8+)</code>
8	<code>default void afficher() {</code>
9	<code>System.out.println("Objet dessinable");</code>
10	<code>}</code>
11	<code>// Méthode statique (Java 8+)</code>
12	<code>static void info() {</code>
13	<code>System.out.println("Interface Dessinable");</code>
14	<code>}</code>
15	<code>}</code>
16	<code>public class Cercle implements Dessinable {</code>
17	<code>private double rayon;</code>
18	<code>public Cercle(double rayon) {</code>
19	<code>this.rayon = rayon;</code>
20	<code>}</code>
21	<code>@Override</code>
22	<code>public void dessiner() {</code>
23	<code>System.out.println("Dessine un cercle de rayon " + rayon);</code>
24	<code>}</code>
25	<code>@Override</code>
26	<code>public void effacer() {</code>
27	<code>System.out.println("Efface le cercle");</code>
28	<code>}</code>
29	<code>// Peut redéfinir la méthode par défaut</code>

```
30     @Override
31     public void afficher() {
32         System.out.println("Cercle de rayon " + rayon);
33     }
34 }
```

■ 7.2 Interfaces multiples

JAVA	Exemple de code
1	public interface Volant {
2	void voler();
3	}
4	public interface Nageant {
5	void nager();
6	}
7	public class Canard implements Volant, Nageant {
8	@Override
9	public void voler() {
10	System.out.println("Le canard vole");
11	}
12	@Override
13	public void nager() {
14	System.out.println("Le canard nage");
15	}
16	}

■ 7.3 Interfaces fonctionnelles

JAVA

Exemple de code

```
1  @FunctionalInterface
2  public interface Operation {
3  int calculer(int a, int b);
4  }
5  // Implémentation classique
6  Operation addition = new Operation() {
7  @Override
8  public int calculer(int a, int b) {
9  return a + b;
10 }
11 };
12 // Lambda expression (Java 8+)
13 Operation multiplication = (a, b) -> a * b;
14 // Utilisation
15 System.out.println(addition.calculer(5, 3)); // 8
16 System.out.println(multiplication.calculer(5, 3)); // 15
```

8. Classes abstraites

■ 8.1 Différences interface vs classe abstraite

Critère	Interface	Classe abstraite
Méthodes	Abstraites + défaut + statiques	Abstraites + concrètes
Attributs	Constantes uniquement	Tous types d'attributs
Héritage	Multiple (implements)	Simple (extends)
Constructeurs	Non	Oui
Usage	Contrat, comportement	Hiérarchie, code commun

■ 8.2 Exemple complet

JAVA	Exemple de code
1	public abstract class Forme {
2	protected String couleur;
3	public Forme(String couleur) {
4	this.couleur = couleur;
5	}
6	// Méthode abstraite
7	public abstract double calculerAire();
8	public abstract double calculerPerimetre();
9	// Méthode concrète
10	public void afficherInfo() {
11	System.out.println("Forme de couleur " + couleur);
12	System.out.println("Aire: " + calculerAire());
13	System.out.println("Périmètre: " + calculerPerimetre());
14	}
15	}
16	public class Rectangle extends Forme {
17	private double longueur;
18	private double largeur;
19	public Rectangle(String couleur, double longueur, double largeur) {
20	super(couleur);
21	this.longueur = longueur;
22	this.largeur = largeur;
23	}
24	@Override
25	public double calculerAire() {
26	return longueur * largeur;
27	}
28	@Override
29	public double calculerPerimetre() {

```
30     return 2 * (longueur + largeur);
31 }
32 }
33 public class Cercle extends Forme {
34     private double rayon;
35     public Cercle(String couleur, double rayon) {
36         super(couleur);
37         this.rayon = rayon;
38     }
39     @Override
40     public double calculerAire() {
41         return Math.PI * rayon * rayon;
42     }
43     @Override
44     public double calculerPerimetre() {
45         return 2 * Math.PI * rayon;
46     }
47 }
```

9. Exceptions

■ 9.1 Hiérarchie des exceptions

JAVA	Exemple de code
1	Throwable
2	■ ■ ■ ■ Error (erreurs système, ne pas attraper)
3	■ ■ ■ ■ OutOfMemoryError
4	■ ■ ■ ■ StackOverflowError
5	■ ■ ■ ■ Exception
6	■ ■ ■ ■ RuntimeException (non-vérifiées)
7	■ ■ ■ ■ NullPointerException
8	■ ■ ■ ■ IndexOutOfBoundsException
9	■ ■ ■ ■ IllegalArgumentException
10	■ ■ ■ ■ ArithmeticException
11	■ ■ ■ ■ IOException (vérifiées)
12	■ ■ ■ ■ FileNotFoundException
13	■ ■ ■ ■ SQLException

■ 9.2 Gestion des exceptions

JAVA	Exemple de code
1	public class GestionExceptions {
2	public static void diviser(int a, int b) {
3	try {
4	int resultat = a / b;
5	System.out.println("Résultat: " + resultat);
6	} catch (ArithmeticException e) {
7	System.err.println("Erreur: Division par zéro");
8	e.printStackTrace();
9	} finally {
10	System.out.println("Bloc finally toujours exécuté");
11	}
12	}
13	// Multi-catch (Java 7+)
14	public static void lireFichier(String chemin) {
15	try {
16	BufferedReader reader = new BufferedReader(new FileReader(chemin));
17	String ligne = reader.readLine();
18	System.out.println(ligne);
19	reader.close();
20	} catch (FileNotFoundException NullPointerException e) {
21	System.err.println("Fichier introuvable ou null");
22	} catch (IOException e) {
23	System.err.println("Erreur de lecture");
24	}
25	}
26	// Try-with-resources (Java 7+)
27	public static void lireFichierAutomatique(String chemin) throws IOException {
28	try (BufferedReader reader = new BufferedReader(new FileReader(chemin))) {
29	String ligne;


```
30 while ((ligne = reader.readLine()) != null) {  
31     System.out.println(ligne);  
32 }  
33 } // reader.close() appelé automatiquement  
34 }  
35 }
```

■ 9.3 Exceptions personnalisées

JAVA	Exemple de code
1	public class SoldeInsuffisantException extends Exception {
2	private double montant;
3	private double solde;
4	public SoldeInsuffisantException(double montant, double solde) {
5	super(String.format("Impossible de retirer %.2f€ (solde: %.2f€)",
6	montant, solde));
7	this.montant = montant;
8	this.solde = solde;
9	}
10	public double getMontant() { return montant; }
11	public double getSolde() { return solde; }
12	}
13	public class CompteBancaire {
14	private double solde;
15	public void retirer(double montant) throws SoldeInsuffisantException {
16	if (montant > solde) {
17	throw new SoldeInsuffisantException(montant, solde);
18	}
19	solde -= montant;
20	}
21	}
22	// Utilisation
23	try {
24	compte.retirer(1000);
25	} catch (SoldeInsuffisantException e) {
26	System.err.println(e.getMessage());
27	System.err.println("Solde disponible: " + e.getSolde());
28	}

10. Collections (List, Set, Map)

■ 10.1 Interface Collection

JAVA	Exemple de code
1	<code>Collection<E></code>
2	■ ■ ■ <code>List<E></code> (ordonnée, doublons autorisés)
3	■ ■ ■ ■ <code>ArrayList<E></code>
4	■ ■ ■ ■ <code>LinkedList<E></code>
5	■ ■ ■ ■ <code>Vector<E></code>
6	■ ■ ■ <code>Set<E></code> (pas d'ordre, pas de doublons)
7	■ ■ ■ ■ <code>HashSet<E></code>
8	■ ■ ■ ■ <code>LinkedHashSet<E></code>
9	■ ■ ■ ■ <code>TreeSet<E></code>
10	■ ■ ■ <code>Queue<E></code>
11	■ ■ ■ <code>PriorityQueue<E></code>
12	■ ■ ■ <code>Deque<E></code>
13	■ ■ ■ <code>ArrayDeque<E></code>
14	<code>Map<K,V></code> (paires clé-valeur)
15	■ ■ ■ <code>HashMap<K,V></code>
16	■ ■ ■ <code>LinkedHashMap<K,V></code>
17	■ ■ ■ <code>TreeMap<K,V></code>
18	■ ■ ■ <code>Hashtable<K,V></code>

■ 10.2 List - Listes

JAVA	Exemple de code
1	<code>import java.util.*;</code>
2	<code>// ArrayList - Tableau dynamique (accès rapide par index)</code>
3	<code>List<String> arrayList = new ArrayList<>();</code>
4	<code>arrayList.add("Java");</code>
5	<code>arrayList.add("Python");</code>
6	<code>arrayList.add("C++");</code>
7	<code>arrayList.add(1, "JavaScript"); // Insertion à l'index 1</code>
8	<code>System.out.println(arrayList.get(0)); // "Java"</code>
9	<code>arrayList.remove(2); // Supprime "Python"</code>
10	<code>// LinkedList - Liste chaînée (insertion/suppression rapides)</code>
11	<code>List<Integer> linkedList = new LinkedList<>();</code>
12	<code>linkedList.add(10);</code>
13	<code>linkedList.add(20);</code>
14	<code>linkedList.add(0, 5); // Ajoute en tête</code>
15	<code>// Parcours</code>
16	<code>for (String langage : arrayList) {</code>
17	<code>System.out.println(langage);</code>
18	<code>}</code>
19	<code>// Itérateur</code>
20	<code>Iterator<Integer> it = linkedList.iterator();</code>
21	<code>while (it.hasNext()) {</code>
22	<code>System.out.println(it.next());</code>
23	<code>}</code>

■ 10.3 Set - Ensembles

JAVA

Exemple de code

```
1 // HashSet - Pas d'ordre, performances O(1)
2 Set<String> hashSet = new HashSet<>();
3 hashSet.add("Ali");
4 hashSet.add("Sami");
5 hashSet.add("Ali"); // Ignoré (doublon)
6 System.out.println(hashSet.size()); // 2
7 // LinkedHashSet - Préserve l'ordre d'insertion
8 Set<String> linkedHashSet = new LinkedHashSet<>();
9 linkedHashSet.add("C");
10 linkedHashSet.add("A");
11 linkedHashSet.add("B");
12 // Itération dans l'ordre: C, A, B
13 // TreeSet - Ordre naturel (ou comparateur)
14 Set<Integer> treeSet = new TreeSet<>();
15 treeSet.add(30);
16 treeSet.add(10);
17 treeSet.add(20);
18 // Itération triée: 10, 20, 30
19 // Opérations ensemblistes
20 Set<Integer> set1 = new HashSet<>(Arrays.asList(1, 2, 3, 4));
21 Set<Integer> set2 = new HashSet<>(Arrays.asList(3, 4, 5, 6));
22 Set<Integer> union = new HashSet<>(set1);
23 union.addAll(set2); // {1, 2, 3, 4, 5, 6}
24 Set<Integer> intersection = new HashSet<>(set1);
25 intersection.retainAll(set2); // {3, 4}
26 Set<Integer> difference = new HashSet<>(set1);
27 difference.removeAll(set2); // {1, 2}
```

■ 10.4 Map - Dictionnaires

JAVA	Exemple de code
1	// HashMap - Pas d'ordre
2	Map<String, Integer> hashMap = new HashMap<>();
3	hashMap.put("Ali", 25);
4	hashMap.put("Sami", 30);
5	hashMap.put("Zaki", 22);
6	System.out.println(hashMap.get("Ali")); // 25
7	System.out.println(hashMap.containsKey("Sami")); // true
8	hashMap.remove("Zaki");
9	// Parcours
10	for (Map.Entry<String, Integer> entry : hashMap.entrySet()) {
11	System.out.println(entry.getKey() + ": " + entry.getValue());
12	}
13	// LinkedHashMap - Préserve l'ordre d'insertion
14	Map<String, String> linkedHashMap = new LinkedHashMap<>();
15	linkedHashMap.put("un", "one");
16	linkedHashMap.put("deux", "two");
17	linkedHashMap.put("trois", "three");
18	// TreeMap - Ordre naturel des clés
19	Map<Integer, String> treeMap = new TreeMap<>();
20	treeMap.put(3, "trois");
21	treeMap.put(1, "un");
22	treeMap.put(2, "deux");
23	// Itération triée par clé: 1, 2, 3
24	// Méthodes utiles
25	hashMap.putIfAbsent("Mohamed", 28);
26	hashMap.getDefault("Inconnu", 0);
27	hashMap.computeIfAbsent("Ahmed", k -> k.length());

■ 10.5 Exemple complet avec Patient

JAVA	Exemple de code
1	<code>import java.util.*;</code>
2	<code>public class GestionPatients {</code>
3	<code>public static void main(String[] args) {</code>
4	<code>// List de patients</code>
5	<code>List<Patient> patients = new ArrayList<>();</code>
6	<code>patients.add(new Patient(1, "Ahmed", "Ali", 1001));</code>
7	<code>patients.add(new Patient(2, "Sami", "Mohamed", 1002));</code>
8	<code>patients.add(new Patient(3, "Zaki", "Ahmed", 1003));</code>
9	<code>// Tri par nom</code>
10	<code>Collections.sort(patients, (p1, p2) -> p1.getNom().compareTo(p2.getNom()));</code>
11	<code>// Set pour éviter les doublons (nécessite equals/hashCode)</code>
12	<code>Set<Patient> patientsUniques = new HashSet<>(patients);</code>
13	<code>// Map: CIN -> Patient</code>
14	<code>Map<Integer, Patient> patientsByCin = new HashMap<>();</code>
15	<code>for (Patient p : patients) {</code>
16	<code>patientsByCin.put(p.getCin(), p);</code>
17	<code>}</code>
18	<code>// Recherche rapide</code>
19	<code>Patient found = patientsByCin.get(2);</code>
20	<code>System.out.println(found);</code>
21	<code>}</code>
22	<code>}</code>

11. Interfaces fonctionnelles & Lambda

■ 11.1 Interfaces fonctionnelles prédéfinies

JAVA	Exemple de code
1	<code>import java.util.function.*;</code>
2	<code>// Predicate<T> : T -> boolean (test)</code>
3	<code>Predicate<Integer> estPositif = x -> x > 0;</code>
4	<code>Predicate<String> estVide = String::isEmpty;</code>
5	<code>System.out.println(estPositif.test(5)); // true</code>
6	<code>System.out.println(estVide.test("hello")); // false</code>
7	<code>// Function<T, R> : T -> R (transformation)</code>
8	<code>Function<String, Integer> longueur = s -> s.length();</code>
9	<code>Function<Integer, String> toString = Object::toString;</code>
10	<code>System.out.println(longueur.apply("hello")); // 5</code>
11	<code>// Consumer<T> : T -> void (action)</code>
12	<code>Consumer<String> afficher = s -> System.out.println(s);</code>
13	<code>Consumer<String> print = System.out::println;</code>
14	<code>afficher.accept("Hello"); // Affiche "Hello"</code>
15	<code>// Supplier<T> : () -> T (fournisseur)</code>
16	<code>Supplier<Double> random = Math::random;</code>
17	<code>Supplier<Date> now = Date::new;</code>
18	<code>System.out.println(random.get());</code>
19	<code>// BiFunction<T, U, R> : (T, U) -> R</code>
20	<code>BiFunction<Integer, Integer, Integer> somme = (a, b) -> a + b;</code>
21	<code>System.out.println(somme.apply(5, 3)); // 8</code>
22	<code>// UnaryOperator<T> : T -> T</code>
23	<code>UnaryOperator<Integer> carre = x -> x * x;</code>
24	<code>System.out.println(carre.apply(5)); // 25</code>
25	<code>// BinaryOperator<T> : (T, T) -> T</code>
26	<code>BinaryOperator<Integer> max = (a, b) -> a > b ? a : b;</code>
27	<code>System.out.println(max.apply(10, 20)); // 20</code>
28	<code>// Comparator<T> : (T, T) -> int</code>
29	<code>Comparator<String> parLongueur = (s1, s2) -> s1.length() - s2.length();</code>

```
30 Comparator<String> alphabetique = String::compareTo;
```

■ 11.2 Références de méthodes

JAVA	Exemple de code
1	// 1. Référence à une méthode statique
2	Function<String, Integer> parseInt = Integer::parseInt;
3	System.out.println(parseInt.apply("123")); // 123
4	// 2. Référence à une méthode d'instance d'un objet particulier
5	String prefix = "Hello, ";
6	Function<String, String> greeter = prefix::concat;
7	System.out.println(greeter.apply("World")); // "Hello, World"
8	// 3. Référence à une méthode d'instance d'un type
9	Function<String, String> upper = String::toUpperCase;
10	System.out.println(upper.apply("java")); // "JAVA"
11	// 4. Référence à un constructeur
12	Function<String, StringBuilder> builder = StringBuilder::new;
13	StringBuilder sb = builder.apply("test");

■ 11.3 Composition de fonctions

JAVA	Exemple de code
1	<code>Function<Integer, Integer> multiplierPar2 = x -> x * 2;</code>
2	<code>Function<Integer, Integer> ajouter10 = x -> x + 10;</code>
3	<code>// Composition: appliquer d'abord multiplierPar2, puis ajouter10</code>
4	<code>Function<Integer, Integer> compose = multiplierPar2.andThen(ajouter10);</code>
5	<code>System.out.println(compose.apply(5)); // (5 * 2) + 10 = 20</code>
6	<code>// Composition inverse: d'abord ajouter10, puis multiplierPar2</code>
7	<code>Function<Integer, Integer> compose2 = multiplierPar2.compose(ajouter10);</code>
8	<code>System.out.println(compose2.apply(5)); // (5 + 10) * 2 = 30</code>
9	<code>// Prédicats</code>
10	<code>Predicate<Integer> estPositif = x -> x > 0;</code>
11	<code>Predicate<Integer> estPair = x -> x % 2 == 0;</code>
12	<code>Predicate<Integer> estPositifEtPair = estPositif.and(estPair);</code>
13	<code>Predicate<Integer> estPositifOuPair = estPositif.or(estPair);</code>
14	<code>Predicate<Integer> estNegatif = estPositif.negate();</code>
15	<code>System.out.println(estPositifEtPair.test(4)); // true</code>
16	<code>System.out.println(estPositifOuPair.test(-2)); // true</code>
17	<code>System.out.println(estNegatif.test(-5)); // true</code>

12. API Stream

■ 12.1 Création de streams

JAVA	Exemple de code
1	// Depuis une collection
2	List<Integer> liste = Arrays.asList(1, 2, 3, 4, 5);
3	Stream<Integer> stream1 = liste.stream();
4	Stream<Integer> parallelStream = liste.parallelStream();
5	// Depuis un tableau
6	int[] tableau = {1, 2, 3, 4, 5};
7	IntStream stream2 = Arrays.stream(tableau);
8	// De valeurs
9	Stream<String> stream3 = Stream.of("a", "b", "c");
10	// Stream vide
11	Stream<Object> stream4 = Stream.empty();
12	// Stream infini
13	Stream<Integer> infini = Stream.iterate(0, n -> n + 1);
14	Stream<Double> random = Stream.generate(Math::random);
15	// Range
16	IntStream range = IntStream.range(1, 10); // 1 à 9
17	IntStream rangeClosed = IntStream.rangeClosed(1, 10); // 1 à 10

■ 12.2 Opérations intermédiaires (lazy)

JAVA	Exemple de code
1	<code>List<Integer> nombres = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);</code>
2	<code>nombres.stream()</code>
3	<code>.filter(n -> n % 2 == 0) // Garder les pairs</code>
4	<code>.map(n -> n * n) // Mettre au carré</code>
5	<code>.sorted() // Trier</code>
6	<code>.distinct() // Supprimer doublons</code>
7	<code>.limit(3) // Garder 3 premiers</code>
8	<code>.skip(1) // Sauter 1er</code>
9	<code>.peek(System.out::println) // Effet de bord (debug)</code>
10	<code>.forEach(System.out::println); // Terminal</code>
11	<code>// flatMap : aplatir une structure</code>
12	<code>List<List<Integer>> listes = Arrays.asList(</code>
13	<code>Arrays.asList(1, 2),</code>
14	<code>Arrays.asList(3, 4),</code>
15	<code>Arrays.asList(5, 6)</code>
16	<code>);</code>
17	<code>List<Integer> aplati = listes.stream()</code>
18	<code>.flatMap(List::stream)</code>
19	<code>.collect(Collectors.toList());</code>
20	<code>// Résultat: [1, 2, 3, 4, 5, 6]</code>

■ 12.3 Opérations terminales (eager)

JAVA	Exemple de code
1	<code>List<Integer> nombres = Arrays.asList(1, 2, 3, 4, 5);</code>
2	<code>// forEach : effet de bord</code>
3	<code>nombres.stream().forEach(System.out::println);</code>
4	<code>// count : compter</code>
5	<code>long count = nombres.stream().filter(n -> n > 2).count(); // 3</code>
6	<code>// collect : collecter dans une structure</code>
7	<code>List<Integer> doubles = nombres.stream()</code>
8	<code>.map(n -> n * 2)</code>
9	<code>.collect(Collectors.toList());</code>
10	<code>Set<Integer> set = nombres.stream().collect(Collectors.toSet());</code>
11	<code>String joined = nombres.stream()</code>
12	<code>.map(String::valueOf)</code>
13	<code>.collect(Collectors.joining(", ")); // "1, 2, 3, 4, 5"</code>
14	<code>// reduce : réduction</code>
15	<code>int somme = nombres.stream().reduce(0, (a, b) -> a + b); // 15</code>
16	<code>int produit = nombres.stream().reduce(1, (a, b) -> a * b); // 120</code>
17	<code>Optional<Integer> max = nombres.stream().reduce(Integer::max);</code>
18	<code>Optional<Integer> min = nombres.stream().reduce(Integer::min);</code>
19	<code>// Méthodes terminales spécialisées</code>
20	<code>int sum = nombres.stream().mapToInt(Integer::intValue).sum();</code>
21	<code>OptionalDouble average =</code>
22	<code>nombres.stream().mapToInt(Integer::intValue).average();</code>
23	<code>IntSummaryStatistics stats = nombres.stream()</code>
24	<code>.mapToInt(Integer::intValue)</code>
25	<code>.summaryStatistics();</code>
26	<code>// anyMatch, allMatch, noneMatch</code>
27	<code>boolean existePositif = nombres.stream().anyMatch(n -> n > 0);</code>
28	<code>boolean tousPositifs = nombres.stream().allMatch(n -> n > 0);</code>
29	<code>boolean aucunNegatif = nombres.stream().noneMatch(n -> n < 0);</code>
	<code>// findFirst, findAny</code>

```
30 Optional<Integer> premier = nombres.stream().findFirst();  
31 Optional<Integer> quelconque = nombres.stream().findAny();
```

■ 12.4 Collectors avancés

JAVA	Exemple de code
1	List<Patient> patients = Arrays.asList(2 new Patient(1, "Ahmed", "Ali", 1001), 3 new Patient(2, "Sami", "Mohamed", 1002), 4 new Patient(3, "Zaki", "Ahmed", 1003), 5 new Patient(4, "Omar", "Ali", 1004) 6); 7 // Grouper par nom 8 Map<String, List<Patient>> parNom = patients.stream() 9 .collect(Collectors.groupingBy(Patient::getNom)); 10 // Compter par nom 11 Map<String, Long> comptesParNom = patients.stream() 12 .collect(Collectors.groupingBy(Patient::getNom, Collectors.counting())); 13 // Partitionner (true/false) 14 Map<Boolean, List<Patient>> partitioned = patients.stream() 15 .collect(Collectors.partitioningBy(p -> p.getCin() > 2)); 16 // Statistiques 17 IntSummaryStatistics stats = patients.stream() 18 .collect(Collectors.summarizingInt(Patient::getCin)); 19 // Joindre 20 String noms = patients.stream() 21 .map(Patient::getNom) 22 .collect(Collectors.joining(", ", "[", "]")); 23 // Résultat: "[Ahmed, Sami, Zaki, Omar]" 24 // toMap 25 Map<Integer, String> mapCinNom = patients.stream() 26 .collect(Collectors.toMap(27 Patient::getCin, 28 Patient::getNom 29));

■ 12.5 Exemple complet de traitement

JAVA	Exemple de code
1	public class AnalyseVentes {
2	public static void main(String[] args) {
3	List<Vente> ventes = chargerVentes();
4	// Top 5 des produits par chiffre d'affaires
5	List<String> top5 = ventes.stream()
6	.collect(Collectors.groupingBy(
7	Vente::getProduit,
8	Collectors.summingDouble(v -> v.getQuantite() * v.getPrix())
9	))
10	.entrySet().stream()
11	.sorted(Map.Entry.<String, Double>comparingByValue().reversed())
12	.limit(5)
13	.map(Map.Entry::getKey)
14	.collect(Collectors.toList());
15	// CA mensuel
16	Map<YearMonth, Double> caParMois = ventes.stream()
17	.collect(Collectors.groupingBy(
18	v -> YearMonth.from(v.getDate()),
19	Collectors.summingDouble(v -> v.getQuantite() * v.getPrix())
20	));
21	// Statistiques globales
22	DoubleSummaryStatistics stats = ventes.stream()
23	.mapToDouble(v -> v.getQuantite() * v.getPrix())
24	.summaryStatistics();
25	System.out.println("CA moyen: " + stats.getAverage());
26	System.out.println("CA max: " + stats.getMax());
27	System.out.println("Nombre de ventes: " + stats.getCount());
28	}
29	}

13. Patterns courants

■ 13.1 Singleton

But : Garantir qu'une classe n'a qu'une seule instance.

JAVA	Exemple de code
1	// Version simple (non thread-safe)
2	public class Database {
3	private static Database instance;
4	private Database() {
5	// Constructeur privé
6	}
7	public static Database getInstance() {
8	if (instance == null) {
9	instance = new Database();
10	}
11	return instance;
12	}
13	}
14	// Version thread-safe (double-checked locking)
15	public class DatabaseThreadSafe {
16	private static volatile DatabaseThreadSafe instance;
17	private DatabaseThreadSafe() {}
18	public static DatabaseThreadSafe getInstance() {
19	if (instance == null) {
20	synchronized (DatabaseThreadSafe.class) {
21	if (instance == null) {
22	instance = new DatabaseThreadSafe();
23	}
24	}
25	}
26	return instance;
27	}
28	}
29	// Version enum (recommandée)

```
30     public enum DatabaseEnum {
31         INSTANCE;
32         public void connect() {
33             System.out.println("Connexion à la base de données");
34         }
35         public void disconnect() {
36             System.out.println("Déconnexion de la base de données");
37         }
38     }
39     // Utilisation
40     DatabaseEnum.INSTANCE.connect();
```

■ 13.2 Factory

But : Créer des objets sans exposer la logique de création au client.

JAVA	Exemple de code
1	public interface Transport {
2	void deplacer();
3	}
4	public class Voiture implements Transport {
5	@Override
6	public void deplacer() {
7	System.out.println("Déplacement en voiture");
8	}
9	}
10	public class Moto implements Transport {
11	@Override
12	public void deplacer() {
13	System.out.println("Déplacement en moto");
14	}
15	}
16	public class Velo implements Transport {
17	@Override
18	public void deplacer() {
19	System.out.println("Déplacement en vélo");
20	}
21	}
22	public class TransportFactory {
23	public static Transport creerTransport(String type) {
24	return switch (type.toLowerCase()) {
25	case "voiture" -> new Voiture();
26	case "moto" -> new Moto();
27	case "velo" -> new Velo();
28	default -> throw new IllegalArgumentException("Type inconnu: " + type);
29	};


```
30     }
31     }
32     // Utilisation
33     Transport t1 = TransportFactory.creerTransport("voiture");
34     Transport t2 = TransportFactory.creerTransport("moto");
35     t1.deplacer(); // "Déplacement en voiture"
36     t2.deplacer(); // "Déplacement en moto"
```

■ 13.3 Strategy

But : Définir une famille d'algorithmes et les rendre interchangeables.

JAVA	Exemple de code
1	public interface StrategieTri {
2	void trier(int[] tableau);
3	}
4	public class TriRapide implements StrategieTri {
5	@Override
6	public void trier(int[] tableau) {
7	// Implémentation du tri rapide
8	Arrays.sort(tableau);
9	System.out.println("Trié avec tri rapide");
10	}
11	}
12	public class TriBulles implements StrategieTri {
13	@Override
14	public void trier(int[] tableau) {
15	// Implémentation du tri à bulles
16	for (int i = 0; i < tableau.length - 1; i++) {
17	for (int j = 0; j < tableau.length - i - 1; j++) {
18	if (tableau[j] > tableau[j + 1]) {
19	int temp = tableau[j];
20	tableau[j] = tableau[j + 1];
21	tableau[j + 1] = temp;
22	}
23	}
24	}
25	System.out.println("Trié avec tri à bulles");
26	}
27	}
28	public class ContexteTri {
29	private StrategieTri strategie;

```
30     public ContexteTri(StrategieTri strategie) {
31         this.strategie = strategie;
32     }
33     public void setStrategie(StrategieTri strategie) {
34         this.strategie = strategie;
35     }
36     public void executerTri(int[] tableau) {
37         strategie.trier(tableau);
38     }
39 }
40 // Utilisation
41 int[] nombres = {5, 2, 8, 1, 9};
42 ContexteTri contexte = new ContexteTri(new TriRapide());
43 contexte.executerTri(nombres);
44 // Changer de stratégie à la volée
45 contexte.setStrategie(new TriBulles());
46 contexte.executerTri(nombres);
```

■ 13.4 Observer

But : Notifier automatiquement plusieurs objets lorsqu'un objet change d'état.

JAVA	Exemple de code
1	<code>import java.util.ArrayList;</code>
2	<code>import java.util.List;</code>
3	<code>public interface Observer {</code>
4	<code>void update(String message);</code>
5	<code>}</code>
6	<code>public class Subject {</code>
7	<code>private List<Observer> observers = new ArrayList<>();</code>
8	<code>private String state;</code>
9	<code>public void attach(Observer observer) {</code>
10	<code>observers.add(observer);</code>
11	<code>}</code>
12	<code>public void detach(Observer observer) {</code>
13	<code>observers.remove(observer);</code>
14	<code>}</code>
15	<code>public void setState(String state) {</code>
16	<code>this.state = state;</code>
17	<code>notifyObservers();</code>
18	<code>}</code>
19	<code>private void notifyObservers() {</code>
20	<code>for (Observer observer : observers) {</code>
21	<code>observer.update(state);</code>
22	<code>}</code>
23	<code>}</code>
24	<code>}</code>
25	<code>public class ConcreteObserver implements Observer {</code>
26	<code>private String name;</code>
27	<code>public ConcreteObserver(String name) {</code>
28	<code>this.name = name;</code>
29	<code>}</code>

```
30     @Override
31     public void update(String message) {
32         System.out.println(name + " a reçu: " + message);
33     }
34 }
35 // Utilisation
36 Subject subject = new Subject();
37 Observer obs1 = new ConcreteObserver("Observateur 1");
38 Observer obs2 = new ConcreteObserver("Observateur 2");
39 subject.attach(obs1);
40 subject.attach(obs2);
41 subject.setState("Nouvel état!");
42 // Affiche:
43 // Observateur 1 a reçu: Nouvel état!
44 // Observateur 2 a reçu: Nouvel état!
```

■ 13.5 Builder

But : Construire des objets complexes étape par étape.

JAVA	Exemple de code
1	public class Voiture {
2	private final String marque;
3	private final String modele;
4	private final int annee;
5	private final String couleur;
6	private final boolean gps;
7	private final boolean climatisation;
8	private Voiture(VoitureBuilder builder) {
9	this.marque = builder.marque;
10	this.modele = builder.modele;
11	this.annee = builder.annee;
12	this.couleur = builder.couleur;
13	this.gps = builder.gps;
14	this.climatisation = builder.climatisation;
15	}
16	public static class VoitureBuilder {
17	private final String marque;
18	private final String modele;
19	private int annee;
20	private String couleur = "Blanc";
21	private boolean gps = false;
22	private boolean climatisation = false;
23	public VoitureBuilder(String marque, String modele) {
24	this.marque = marque;
25	this.modele = modele;
26	}
27	public VoitureBuilder annee(int annee) {
28	this.annee = annee;
29	return this;

```
30     }
31     public VoitureBuilder couleur(String couleur) {
32         this.couleur = couleur;
33         return this;
34     }
35     public VoitureBuilder avecGPS() {
36         this.gps = true;
37         return this;
38     }
39     public VoitureBuilder avecClimatisation() {
40         this.climatisation = true;
41         return this;
42     }
43     public Voiture build() {
44         return new Voiture(this);
45     }
46     }
47     @Override
48     public String toString() {
49         return String.format("%s %s (%d) - Couleur: %s, GPS: %b, Clim: %b",
50             marque, modele, annee, couleur, gps, climatisation);
51     }
52     }
53     // Utilisation
54     Voiture voiture = new Voiture.VoitureBuilder("Toyota", "Corolla")
55         .annee(2024)
56         .couleur("Rouge")
57         .avecGPS()
58         .avecClimatisation()
59         .build();
60     System.out.println(voiture);
```


Conclusion

Ce guide couvre les **concepts essentiels** de la programmation orientée objet en Java. Les points clés à retenir :

■ ■ Les 4 piliers de la POO

- **Encapsulation** : Masquer les détails, exposer une interface contrôlée
- **Héritage** : Réutiliser et spécialiser le code
- **Polymorphisme** : Comportements différents selon le contexte
- **Abstraction** : Définir des contrats sans tous les détails

■ ■ Concepts avancés

- **Collections** : Structures de données puissantes (List, Set, Map)
- **Streams** : Traitement de données de manière déclarative et fonctionnelle
- **Lambda & Interfaces fonctionnelles** : Code concis et expressif
- **Exceptions** : Gestion robuste des erreurs
- **Design Patterns** : Solutions éprouvées aux problèmes récurrents

■ ■ Pour aller plus loin

La maîtrise de Java passe par :

****La pratique régulière**** : Coder quotidiennement

****Les projets personnels**** : Mettre en œuvre les concepts

****La lecture de code**** : Étudier du code open source

****La documentation officielle**** : docs.oracle.com/javase

Bonne chance dans votre apprentissage de Java !

Guide créé pour les étudiants de 3ème année ESPRIT